

Competitive Programming

Section 22 Editorial: One-Dimensional DP

Competitive Programming Course

2026-07-06

Table of contents

1	Editorial overview	4
2	DP reminder	5
3	A. Broken Stairs	6
3.1	Restatement	6
3.2	Constraints and traps	6
3.3	Observations	6
3.4	Sample walkthrough	6
3.5	Algorithm	7
3.6	Correctness proof	7
3.7	Complexity	8
3.8	C++ implementation notes	8
3.9	Python implementation notes	8
3.10	Full C++ solution	8
3.11	Full Python solution	9
3.12	Common wrong answers	9
3.13	Tests that matter	10
4	B. Training Score	11
4.1	Restatement	11
4.2	Constraints and traps	11
4.3	Observations	11
4.4	Sample walkthrough	11
4.5	Algorithm	12
4.6	Correctness proof	12
4.7	Complexity	13
4.8	C++ implementation notes	13
4.9	Python implementation notes	13
4.10	Full C++ solution	13
4.11	Full Python solution	14
4.12	Common wrong answers	14
4.13	Tests that matter	14
5	C. Energy Route	15
5.1	Restatement	15
5.2	Constraints and traps	15
5.3	Observations	15
5.4	Sample walkthrough	16
5.5	Algorithm	16
5.6	Correctness proof	17
5.7	Complexity	17
5.8	C++ implementation notes	17
5.9	Python implementation notes	17

5.10 Full C++ solution	17
5.11 Full Python solution	18
5.12 Common wrong answers	19
5.13 Tests that matter	19
6 Testing and stress strategy	20
7 What to carry forward	21

1 Editorial overview

Problem	DP type	Main recurrence
A. Broken Stairs	counting ways	$dp[i] = dp[i-1] + dp[i-2]$ unless broken
B. Training Score	max take/skip	$dp[i] = \max(dp[i-1], dp[i-2] + a[i])$
C. Energy Route	min over previous states	$dp[i] = \min(dp[j] + cost(j, i))$

The shared workflow:

```
define state
initialize base cases
derive transition from the final move/choice
compute states in dependency order
print the answer state
```

2 DP reminder

A DP array is not magic storage. It is a table of solved subproblems.

For every solution below, pay attention to the exact state sentence:

```
dp[i] = ...
```

The recurrence only makes sense after that sentence is precise.

3 A. Broken Stairs

3.1 Restatement

You start at step 0 and want to reach step n . Each move climbs either 1 or 2 steps. Some steps are broken, and you may not land on them. Count the number of valid ways to reach step n modulo 1_000_000_007.

3.2 Constraints and traps

n can be $2 * 10^5$, so enumerating all paths is impossible.

Important traps:

```
Step 0 is the starting position and is not listed as broken.  
If the target step  $n$  is broken, the answer is 0.  
For counting,  $dp[0]$  should be 1, not 0.  
Modulo is required because the number of paths grows exponentially.  
If  $b = 0$ , the broken-step line is omitted.
```

3.3 Observations

Consider the last move into step i .

If i is not broken, the final move came from:

```
 $i - 1$ , using a one-step move  
 $i - 2$ , using a two-step move
```

These two groups of paths are disjoint, because their last previous step is different. Therefore we add their counts.

If i is broken, no path may land there:

```
 $dp[i] = 0$ 
```

3.4 Sample walkthrough

Sample:

```
n = 5
broken = {3}
```

Compute:

```
dp[0] = 1
dp[1] = dp[0] = 1
dp[2] = dp[1] + dp[0] = 2
dp[3] = 0 because step 3 is broken
dp[4] = dp[3] + dp[2] = 2
dp[5] = dp[4] + dp[3] = 2
```

The two valid routes are:

```
0 -> 1 -> 2 -> 4 -> 5
0 -> 2 -> 4 -> 5
```

3.5 Algorithm

```
read n and b
mark broken steps
dp[0] = 1

for i from 1 to n:
    if i is broken:
        dp[i] = 0
    else:
        dp[i] = dp[i-1]
        if i >= 2:
            dp[i] += dp[i-2]
        dp[i] %= MOD

print dp[n]
```

3.6 Correctness proof

Lemma 1. For every non-broken step $i \geq 1$, every valid path to i ends at either $i-1$ or $i-2$ immediately before reaching i .

Proof. The only allowed move lengths are 1 and 2. Therefore the step before i must be $i-1$ or $i-2$. $\square$

Lemma 2. For every i , $dp[i]$ equals the number of valid ways to reach step i , modulo MOD, after the algorithm computes it.

Proof. By induction on i . For $i = 0$, there is exactly one way to be at the start: make no moves, so $dp[0] = 1$. For $i > 0$, if step i is broken, no valid path may land there, and the algorithm sets $dp[i] = 0$. Otherwise, by Lemma 1, every valid path to i comes from a valid

path to $i-1$ followed by a one-step move, or from a valid path to $i-2$ followed by a two-step move. These cases are disjoint, and by the induction hypothesis their counts are `dp[i-1]` and `dp[i-2]`. The algorithm adds exactly these counts. $\square$

Theorem. The algorithm prints the number of valid ways to reach step n , modulo `1_000_000_007`.

Proof. By Lemma 2 with $i = n$, `dp[n]` has exactly the required meaning. $\square$

3.7 Complexity

```
Time:  O(n + b)
Memory: O(n)
```

The memory can be reduced to $O(1)$ if broken status is streamed or stored in a set, but the array version is clearer and already fits easily.

3.8 C++ implementation notes

Use `long long` for additions before taking modulo. Store broken steps in a `vector<char>` to avoid unnecessary memory.

3.9 Python implementation notes

Use a boolean list for `broken`. If $b = 0$, there are simply no remaining broken-step tokens.

3.10 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    const long long MOD = 1000000007LL;
    int n, b;
    if (!(cin >> n >> b)) return 0;
    vector<char> broken(n + 1, false);
    for (int i = 0; i < b; i++) {
        int x;
        cin >> x;
        broken[x] = true;
    }
```

```

vector<long long> dp(n + 1, 0);
dp[0] = 1;
for (int i = 1; i <= n; i++) {
    if (broken[i]) continue;
    dp[i] = dp[i - 1];
    if (i >= 2) dp[i] = (dp[i] + dp[i - 2]) % MOD;
}
cout << dp[n] % MOD << '\n';
return 0;
}

```

3.11 Full Python solution

```

import sys

MOD = 1_000_000_007

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n, b = data[0], data[1]
    broken = [False] * (n + 1)
    for x in data[2:2 + b]:
        broken[x] = True
    dp = [0] * (n + 1)
    dp[0] = 1
    for i in range(1, n + 1):
        if broken[i]:
            continue
        dp[i] = dp[i - 1]
        if i >= 2:
            dp[i] = (dp[i] + dp[i - 2]) % MOD
    print(dp[n] % MOD)

if __name__ == "__main__":
    main()

```

3.12 Common wrong answers

Using `dp[0] = 0`, which makes every count zero.
 Forgetting that broken step `n` gives answer 0.

Accessing `dp[-1]` accidentally when computing step 1.
Not taking modulo in C++ and overflowing.

3.13 Tests that matter

`n = 1` and step 1 broken $\rightarrow$ 0.
No broken steps $\rightarrow$ Fibonacci-style count.
Consecutive broken steps can block all routes.
Large `n` with no broken steps tests modulo.

4 B. Training Score

4.1 Restatement

There are n days. Training on day i gives $a[i]$ points. You may not train on two consecutive days. You may skip any day, including all days. Find the maximum total score.

4.2 Constraints and traps

n is large, so trying every subset is impossible.

Important traps:

```
Scores may be negative.  
Because skipping all days is allowed, the answer is never below 0.  
Taking day  $i$  forbids day  $i-1$  but does not forbid day  $i-2$ .  
Use long long in C++ because scores can be large.
```

4.3 Observations

Define:

```
dp[i] = maximum score using only the first  $i$  days
```

For the last day in this prefix, either:

```
skip day  $i$ :  
    best score is dp[i-1]  
  
take day  $i$ :  
    day  $i-1$  must be skipped, so best compatible previous score is dp[i-2]  
    total is dp[i-2] + a[i]
```

Take the better option.

4.4 Sample walkthrough

Scores:

```
5 1 2 10 6 2
```

Using one-based day numbers:

```
dp[0] = 0
dp[1] = max(dp[0], dp[-1] + 5) = 5
```

It is cleaner in code to use two previous values instead of accessing `dp[-1]`. The resulting table:

i:	0	1	2	3	4	5	6
dp:	0	5	5	7	15	15	17

One optimal choice is days 1, 4, 6, score $5 + 10 + 2 = 17$.

4.5 Algorithm

Array version:

```
dp[0] = 0
dp[1] = max(0, a[1])

for i from 2 to n:
    dp[i] = max(dp[i-1], dp[i-2] + a[i])

print dp[n]
```

Rolling version:

```
prev2 = 0
prev1 = 0

for each score x:
    cur = max(prev1, prev2 + x)
    prev2 = prev1
    prev1 = cur

print prev1
```

4.6 Correctness proof

Lemma 1. In an optimal solution for the first i days, either day i is skipped, or day i is taken and day $i-1$ is skipped.

Proof. Day i has only two possibilities: skipped or taken. If it is taken, the rule forbids training on day $i-1$. $\square$

Lemma 2. After processing the first i days, $dp[i]$ equals the maximum valid score using only those days.

Proof. By induction. $dp[0] = 0$ is correct because no days are available. For $i > 0$, Lemma 1 splits every valid optimal solution into two cases. If day i is skipped, the best possible score is $dp[i-1]$. If day i is taken, the best compatible score from earlier days is $dp[i-2]$, plus $a[i]$. By the induction hypothesis, those smaller DP values are optimal. The algorithm takes the maximum of exactly these two exhaustive cases. $\square$

Theorem. The algorithm prints the maximum possible training score.

Proof. By Lemma 2 with $i = n$, $dp[n]$ is the best score using all days. The rolling implementation stores the same recurrence values, only without the full table. $\square$

4.7 Complexity

```
Time:  $O(n)$ 
Memory:  $O(1)$  with rolling variables
```

4.8 C++ implementation notes

Initialize both previous values to 0. This naturally handles negative scores because skipping all days remains available.

4.9 Python implementation notes

The rolling version is short and avoids allocating an array. Use it only after the recurrence is clear.

4.10 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    long long prev2 = 0, prev1 = 0;
    for (int i = 0; i < n; i++) {
        long long x;
        cin >> x;
        long long cur = max(prev1, prev2 + x);
```

```

        prev2 = prev1;
        prev1 = cur;
    }
    cout << prev1 << '\n';
    return 0;
}

```

4.11 Full Python solution

```

import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n = data[0]
    a = data[1:1 + n]
    prev2 = 0
    prev1 = 0
    for x in a:
        cur = max(prev1, prev2 + x)
        prev2, prev1 = prev1, cur
    print(prev1)

if __name__ == "__main__":
    main()

```

4.12 Common wrong answers

Forcing at least one day to be selected, which fails on all-negative input.
 Using `dp[i-1] + a[i]`, which allows adjacent selected days.
 Using `int` in C++ for large sums.
 Updating `prev1` before computing `cur` in the rolling version.

4.13 Tests that matter

All scores negative -> answer 0.
 One day -> `max(0, a[1])`.
 Alternating large/small values -> tests take/skip logic.
 Large positive values -> tests long long.

5 C. Energy Route

5.1 Restatement

There are n platforms in a line. Platform i has height $h[i]$. You start on platform 1. From platform i , you may jump forward to any platform up to k positions ahead. The cost of a jump from i to j is $\text{abs}(h[i] - h[j])$. Find the minimum total cost to reach platform n .

5.2 Constraints and traps

n can be large, but $k \leq 50$, so checking up to k previous platforms for each state is acceptable.

Important traps:

```
The graph is acyclic because jumps only go forward.  
The best previous platform for  $i$  is not always  $i-1$ .  
Use a large infinity for uncomputed minimum states.  
The answer for  $n = 1$  is 0.  
Use long long in C++ because heights are large.
```

5.3 Observations

The final jump into platform i must come from some platform j such that:

```
 $i - k \leq j < i$ 
```

If the cheapest cost to reach j is already known, then using j as the final previous platform costs:

```
 $\text{dp}[j] + \text{abs}(h[j] - h[i])$ 
```

Take the minimum over all valid j .

5.4 Sample walkthrough

Input:

```
5 3
10 30 40 50 20
```

Zero-based table:

```
platform:  0   1   2   3   4
height:    10  30  40  50  20
```

Compute:

```
dp[0] = 0

dp[1] = dp[0] + |10-30| = 20

dp[2] = min(
    dp[0] + |10-40| = 30,
    dp[1] + |30-40| = 30
) = 30

dp[3] = min(
    dp[0] + |10-50| = 40,
    dp[1] + |30-50| = 40,
    dp[2] + |40-50| = 40
) = 40

dp[4] = min(
    dp[1] + |30-20| = 30,
    dp[2] + |40-20| = 50,
    dp[3] + |50-20| = 70
) = 30
```

Answer: 30.

5.5 Algorithm

```
read n, k, heights
dp[0] = 0
dp[i] = INF for all other i

for i from 1 to n-1:
    for j from max(0, i-k) to i-1:
        dp[i] = min(dp[i], dp[j] + abs(h[j] - h[i]))

print dp[n-1]
```

5.6 Correctness proof

Lemma 1. Every valid route to platform $i > 0$ has a unique final previous platform j with $\max(0, i-k) \leq j < i$.

Proof. The last move must be a forward jump ending at i . Since jumps may advance at most k positions, the previous platform j must satisfy $i-k \leq j < i$. That final previous platform is unique for the route. $\square$

Lemma 2. After computing $dp[i]$, it equals the minimum energy needed to reach platform i .

Proof. By induction on i . For $i = 0$, no jump is needed, so $dp[0] = 0$. For $i > 0$, Lemma 1 says every valid route reaches some valid previous platform j , then jumps from j to i . By the induction hypothesis, the minimum energy to reach each such j is $dp[j]$, so the best route with final previous platform j has cost $dp[j] + \text{abs}(h[j] - h[i])$. The algorithm takes the minimum over all valid j , so it finds the best route to i . $\square$

Theorem. The algorithm prints the minimum total energy to reach platform n .

Proof. By Lemma 2 for the last platform, $dp[n-1]$ is the required minimum. $\square$

5.7 Complexity

For each platform, the algorithm checks at most k previous platforms.

```
Time:  $O(nk)$   
Memory:  $O(n)$ 
```

With $k \leq 50$, this is effectively linear for the given constraints.

5.8 C++ implementation notes

Use `llabs` or `abs` with `long long` values. Set `INF` to a value far above the maximum possible route cost, such as `(1LL << 62)`.

5.9 Python implementation notes

The nested loop is fine for $k \leq 50$. Store `hi = h[i]` inside the outer loop if you want to avoid repeated indexing.

5.10 Full C++ solution

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, k;
    if (!(cin >> n >> k)) return 0;
    vector<long long> h(n);
    for (long long &x : h) cin >> x;
    const long long INF = (1LL << 62);
    vector<long long> dp(n, INF);
    dp[0] = 0;
    for (int i = 1; i < n; i++) {
        for (int j = max(0, i - k); j < i; j++) {
            dp[i] = min(dp[i], dp[j] + llabs(h[j] - h[i]));
        }
    }
    cout << dp[n - 1] << '\n';
    return 0;
}

```

5.11 Full Python solution

```

import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n, k = data[0], data[1]
    h = data[2:2 + n]
    inf = 10**30
    dp = [inf] * n
    dp[0] = 0
    for i in range(1, n):
        best = inf
        start = max(0, i - k)
        hi = h[i]
        for j in range(start, i):
            cand = dp[j] + abs(h[j] - hi)
            if cand < best:
                best = cand
        dp[i] = best
    print(dp[-1])

```

```
if __name__ == "__main__":  
    main()
```

5.12 Common wrong answers

Only considering jumps from $i-1$.
Allowing jumps longer than k .
Computing states right-to-left without changing the recurrence.
Using a small INF that can be beaten by real path costs.
Forgetting $n = 1$, where the answer is 0.

5.13 Tests that matter

$k = 1$ -> only one possible route.
 $n = 1$ -> answer 0.
Best route skips several platforms.
Heights near 10^9 -> tests long long.
Several previous platforms tie for best cost.

6 Testing and stress strategy

The section checker runs:

```
CP_TARGET=solution python3 sections/22_one_dimensional_dp/check.py
```

The random tests cover:

Broken Stairs:

random broken sets, no broken steps, and blocked targets.

Training Score:

positive, negative, and mixed scores.

Energy Route:

varied k, single-platform cases, and random heights.

For your own stress tests:

A: compare DP against brute-force recursive enumeration for $n \leq 20$.

B: enumerate all subsets for $n \leq 20$ and reject adjacent selections.

C: run Dijkstra on the DAG for $n \leq 50$ and compare with the DP.

The brute force should be used only as a testing oracle on tiny inputs, not as the submitted solution.

7 What to carry forward

One-dimensional DP is the first reusable DP pattern:

```
last move -> transition  
last choice -> take/skip  
small prefix -> bigger prefix
```

The next sections will add more state dimensions and more complex computation orders. The discipline from this section remains the same: define the state before writing the recurrence.